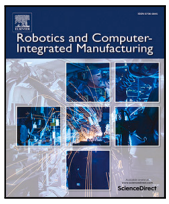
Robotics and Computer–Integrated Manufacturing 96 (2025) 103047



Contents lists available at ScienceDirect

Robotics and Computer-Integrated Manufacturing

journal homepage: www.elsevier.com/locate/rcim



Full length article



Joint optimization of production scheduling and worker allocation under a resource dedication policy in aircraft assembly lines

Xiong Zheng [a], Fei Qiao [a,b], Shiqi You [b], Xi Vincent Wang [c], Lihui Wang [c], Junkai Wang [b,*]

[a] Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, Shanghai, 201210, China
[b] College of Electronics and Information Engineering, Tongji University, Shanghai, 201804, China
[c] Department of Production Engineering, KTH Royal Institute of Technology, Sweden

ARTICLE INFO

Keywords:
Aircraft assembly line
Multi-mode task
Worker allocation
Joint optimization
Resource-makespan mapping

ABSTRACT

Worker resources are crucial in aircraft final assembly lines (AFAL), which are characterized by extensive manual assembly tasks. The features of AFAL, including resource constraints, makespan balancing, and flexibility in resource allocation, present greater challenges than conventional scheduling problems. This paper addresses the joint optimization problem of worker allocation under a resource dedication policy and scheduling of multi-mode tasks in the AFAL. Bi-objective with lexicographic order of minimizing the cycle time and total worker investment is considered, and an integer programming model is developed to formulate this problem. We propose a resource reallocation embedded genetic algorithm (RReGA) to solve this optimization challenge effectively. Initially, hybrid dispatch rules (HDRs) are employed to estimate the resource-makespan mapping of each workstation, yielding a high-quality initial resource allocation solution. Leveraging these mappings, a resource reallocation method, composed of a resource transfer strategy and a resource recovery strategy, is embedded in the evolutionary process of the genetic algorithm (GA) searching for scheduling solutions at the workstation. The resource transfer strategy is responsible for dynamic resource transfer across workstations, following a novel transfer principle to optimize the cycle time; while the resource recovery strategy aims to meet makespan constraints with the fewest workers to minimize cost. The efficacy and superior performance of the proposed algorithm are validated through comprehensive comparison and ablation experiments, as well as an unbalanced case study.

1. Introduction

The aircraft manufacturing industry has experienced rapid growth over the past few decades, transitioning from single production to batch production. As the final stage of aircraft production, the AFAL is responsible for assembling the aircraft fuselage, including the addition of the nose and tail sections, wings, engines, and landing gear. Remarkably, this stage accounts for up to 50% of the total production duration and over 20% of the overall production cost [1]. Consequently, enhancing the efficiency of the aircraft assembly process is of paramount importance in optimizing aircraft production.

The AFAL consists of a specific number of workstations, each with a set of assembly task that must adhere to precedence constraints [2]. All assembly tasks must be completed within a predetermined cycle time before the aircraft can proceed to the subsequent workstation [3]. This necessitates balanced completion times across all workstations. Worker allocation in the AFAL is one of the most significant resource optimization problems since more than 70% of assembly tasks are manually executed. Flexibility is inherent in worker allocation, as the processing time for a given task is directly influenced by the number of workers assigned to it. This allows for the execution of a task in various modes, each corresponding to a certain worker input and processing time [4].

This paper investigates the problem of worker allocation and multi-mode assembly task scheduling in the AFAL, considering lexicographic bi-objective optimization. Specifically, the primary objective is to minimize the workstation cycle time, thereby enhancing the production efficiency. Resource distribution principles are typically divided into two policies: resource sharing (RS) and resource dedication (RD) policy. Distinct from the conventional multi-project scheduling problems [5,6], the ability to rapidly and arbitrarily transfer workers among different workstations during a production cycle is not feasible in the AFAL, thus precluding the adoption of the RS policy. Consequently, the worker allocation problem studied in this paper presumes the RD policy [7]. Under the RD policy, workers are not shared from a common

* Corresponding author.
E-mail address: jkwang@tongji.edu.cn (J. Wang).

https://doi.org/10.1016/j.rcim.2025.103047
Received 7 November 2024; Received in revised form 3 May 2025; Accepted 3 May 2025
Available online 24 May 2025
0736-5845/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

pool but are dedicated in specific amounts to individual workstations throughout a production cycle. The fundamental multi-mode resource constrained project scheduling problem (MRCPSP) is recognized as NP-hard [8]. As an extension of the MRCPSP, the multi-mode resource constrained multi-project scheduling problem (MRCMPSP) under the RS policy can be simplified to a large-scale MRCPSP by integrating all assembly task sets. The problem studied in this paper, namely MRCMPSP under the RD policy, can be viewed as a combination of several coupled MRCPSP. As a result, it is also NP-hard.

The secondary objective is to minimize the total worker investment, aiming to reduce the resource cost [9]. Bao et al. [10] demonstrated that minimizing worker investment helps balance the makespans of the workstations, given the assembly tasks and cycle time. It expands the objective space and could potentially lead to a local optimum. To the best of our knowledge, existing studies concentrating on resource allocation and scheduling under the RD policy in the AFAL have yet to concurrently address the optimization of cycle time and resource cost, which are paradoxical objectives but commonly encountered in practice. Motivated by these challenges and gaps, we propose the RReGA algorithm to address this problem. A mapping-based resource reallocation method, composed of a resource transfer strategy and a resource recovery strategy, is embedded within the evolutionary process, facilitating adaptable modifications in worker allocation without incurring additional computational burdens.

The remainder of this paper is organized as follows. Section 2 gives a literature review. A brief problem description and the mathematical model of the problem are presented in Section 3. A resource-makespan mapping estimation method based on HDRs and the proposed RReGA algorithm is detailed in Section 4. Section 5 reports the computational results and discussions. Finally, Section 6 concludes the paper and discusses topics requiring further research.

2. Literature review

The problem addressed in this paper is defined based on resource allocation and project scheduling, which can be viewed as a MRCMPSP under the RD policy. Therefore, we present the literature review on the resource allocation of AFAL, the scheduling of AFAL, and the MRCMPSP, respectively.

2.1. Resource allocation of aircraft final assembly line

The resource allocation problems manifest in several variants depending on the objectives. Two prevalent types are (i) resource constrained scheduling problem (RCSP) and (ii) resource investment optimization problem (RIOP).

The RCSP involves resource allocation and scheduling under limited resource availability, aiming to minimize certain objectives, typically makespan or total tardiness penalties [11]. Behdad et al. [12] proposed a disassembly sequence planning (DSP) algorithm in the human–robot collaboration (HRC) setting. They considered several factors including limited resources and human workers' safety. Arkhipov et al. [13] addressed labor skills and ergonomic constraints, proposing a joint exact method based on constraint programming (CP) and integer linear programming (ILP). Given the NP-hard nature of RCSP, such exact methods are impractical for large-scale instances. Ren et al. [14] proposed time windows of resources, accounting for varying resource availability over time, and integrated project splitting to reduce makespan and cost. In assembly flow-line scenarios involving resource transfers between workstations, Lu et al. [15] considered resource transfer time based on activity precedence and devised a branch-and-bound embedded genetic algorithm. Fang et al. [16] focused on multi-stage workstations allowing multiple worker transfers within a cycle time, introducing a modified NSGA-IV to minimize workstation cycle time and balance workload across workstations and workers.

The RIOP aims to minimize total resource cost while ensuring timely completion of all assembly tasks [17]. Ağpak and Gökçen [18] proposed a 0–1 integer programming model to balance assembly lines with minimal workstations and resources, emphasizing production flexibility and efficiency, though scalability remains a challenge. Xin et al. [19] addressed the multi-skilled workers assignment problem using an adaptive binary particle swarm optimization (A-BPSO) algorithm to balance workload and minimize labor cost. In low-volume manufacturing, Biele and Mönch [20] explored hybrid approaches to optimize mixed-model assembly lines, focusing on variable flow time policies to reduce overtime and inventory holding cost. They employed heuristic methods to determine the optimal crew size and worker allocation. Acknowledging the uncertainties in the processing times of tasks due to real-world disruptions, Bao et al. [21] proposed a robust optimization approach, ensuring robust decision-making under uncertain conditions. Portoleau et al. [22] advocated for robust decision trees to manage uncertainties in multi-mode scheduling problems, enhancing robustness while optimizing resource investments across decision points. In the AFAL, the assembly task processing time typically depends on the amount of worker resources allocated, which is known as the multi-mode scheduling problem. Borreguero et al. [23] introduced an event-based MILP formulation and a CP model with large-neighborhood search techniques to optimize resource-leveling objectives.

2.2. Scheduling of aircraft final assembly line

The scheduling of AFAL can be abstracted as a multi-project scheduling problem. Given its uniform pace cycle across workstations, the assignment of assembly tasks and line balancing are typically addressed simultaneously. Heike et al. [24] developed linear and nonlinear programming models to evaluate low-volume manufacturing cycle time and worker allocation alternatives. Da Silva et al. [25] investigated the production scheduling of assembly fixtures with adjacency constraints specific to the aeronautical industry. They proposed MILP models tailored to different stages of workforce learning and qualification in the assembly process. To solve the local order scheduling problem for mixed-model AFAL, Buergin et al. [26] introduced an optimization model that integrates mixed-model sequencing and level scheduling techniques, aiming to minimize work overload. The exact algorithms proposed in the studies mentioned above face significant challenges when addressing large-scale instances, whereas heuristic algorithms can deliver high-quality solutions within a reasonable time. Shan et al. [27] proposed an adaptive genetic algorithm tailored for demand-driven scheduling in aircraft assembly. Bao et al. [10] investigated the AFAL balancing problem. They developed a two-stage heuristic approach for task assignment, aiming to balance cycle time and minimize resource cost.

In addition, some artificial intelligence algorithms such as machine learning and reinforcement learning have been applied to the scheduling of aircraft assembly in recent years. Zhu et al. [28] introduced a reactive scheduling method for addressing uncertainties in material delivery. They used the support vector data description (SVDD) for offline training and an online reactive scheduling phase. Yan and Wang [29] tackled the joint decision-making problem of scheduling and flexible preventive maintenance. They innovated with a double-layer Q-learning algorithm, which incorporated maintenance decision-making into the scheduling process. Tereshchuk et al. [30] focused on multi-robot assembly scenarios. They introduced a two-step, data-driven methodology, emphasizing the minimization of tool changeovers to reduce time cost.

2.3. Multi-mode resource constrained multi-project scheduling

The MRCMPSP is critical in various industries where efficient allocation of resources across multiple projects is necessary to optimize projects' completion time and cost. Chen et al. [31] introduced a

staff time coefficient to match tasks with available staff. Their model integrated MILP and dynamic programming to find optimal solutions efficiently. Can and Ulusoy [32] presented a two-stage decomposition method, which decomposed projects into macro-activities and employed 0–1 mathematical programming models to maximize net present value. In the first Multidisciplinary International Scheduling Conference: Theory & Applications (MISTA) 2013 challenge [33], Asta et al. [34] proposed a hybrid approach combining Monte-Carlo tree search, novel neighborhood moves, memetic algorithms, and hyper-heuristic methods to minimize completion time. Geiger [35] introduced a multi-threaded local search algorithm for the MRCMPSP. Their approach incorporated variable neighborhood search (VNS) and iterated local search (ILS) techniques for parallel implementation.

The above studies treat resources as a pool shared by all projects. In practice, the RS policy may not be feasible if projects are geographically distributed or if it is not preferred to share resources between projects. Beşikci et al. [7] pioneered the introduction of the RD policy. They conceptualized the management of resources within the context of MRCMPSP under the RD policy, terming it as the resource dedication problem (RDP). They proposed a GA enhanced with combinatorial auction (CA) and a heuristic algorithm based on Lagrangian relaxation and subgradient optimization to determine resource allocation. Further expanding on the RDP, Beşikci et al. [36] incorporated the distribution of the total budget among different resource types to ascertain the general resource capacities. They developed a two-phase and a monolithic GA to address this extended problem, which is defined as the resource portfolio problem (RPP). These studies on RDP and RPP paid more attention to resource allocation, while ignoring the scheduling of projects.

2.4. Research gaps and contributions of our paper

In summary, existing research on scheduling in the AFAL reveals several deficiencies. To begin with, few studies have adequately addressed the necessary flexibility in resource allocation. For instance, while Bao et al. [37] considered multi-mode scheduling, their focus was confined to a single workstation. Additionally, since the objectives of minimizing cycle time and resource cost often conflict, few studies have simultaneously optimized these two objectives under limited resource constraints [38]. Furthermore, the uniform pace time characteristic of the AFAL necessitates consistent makespans across all workstations to mitigate resource wastage due to idle periods. These gaps significantly restrict the practical application of existing algorithms. This paper comprehensively addresses these challenges by considering the characteristics of the AFAL, including constraints of limited resource, balancing on cycle time, and flexibility in resource allocation. We propose a RReGA algorithm to tackle these issues. The main contributions of this paper are outlined as follows.

(1) A joint optimization problem concerning worker allocation under the RD policy and the scheduling of multi-mode assembly tasks in the AFAL is presented. A lexicographic bi-objective optimization integer programming model, aiming to minimize both cycle time and cost, is constructed for solving small-scale instances.

(2) A resource-makespan mapping is established for facilitating worker allocation. A mapping-based resource reallocation method with two strategies is proposed, so as to achieve dynamic worker allocation. The resource transfer strategy of RReGA effectively balances makespans across all workstations by transferring workers before execution, resulting in an optimized cycle time.

(3) Following the determination of cycle time, a resource recovery strategy is put forward to actively reclaim redundant workers from each workstation, streamlining labor allocation, all while maintaining the optimal cycle time. Furthermore, the resource reallocation is integrated into the evolutionary process without heavy computational overhead. This approach introduces a novel method of economizing resources without compromising productivity.

Table 1
Notations used for problem description.

Notations	Meaning
k	Total number of workstations.
n	Total number of workers.
w_i	The ith workstation.
n_i	The number of workers allocated to workstation w_i.
J_i	The set of assembly tasks for workstation w_i.
m_i	The number of tasks of J_i.
$J_{i,j}$	The jth task of J_i.
$M_{i,j}$	The available execution modes of task $J_{i,j}$.
d_{ijm}	The processing time of task $J_{i,j}$ executed by mode m.
r_{ijm}	The worker requirement of task $J_{i,j}$ executed by mode m.

3. Problem formulation

3.1. Problem description

In this paper, the terms "worker" and "resource" are synonyms, as only the worker resource is considered. They are used interchangeably in the rest of the paper. The notations employed for the problem description are presented in Table 1. The problem addressed in this study encompasses two primary aspects: (1) the allocation of workers to workstations, and (2) the scheduling of assembly tasks at each workstation. Consider a assembly line shop with k workstations, denoted as $W = \{w_1, w_2, \dots, w_k\}$. Each workstation w_i is required to handle a set of assembly tasks represented by $J_i = \{J_{i,1}, J_{i,2}, \dots, J_{i,m_i}\}$, where m_i signifies the number of tasks to be processed at workstation w_i. The assembly tasks for each workstation are predetermined. The scheduling of these tasks must adhere to precedence constraints, i.e., each task can only commence once all its predecessors have been completed. Each assembly task $J_{i,j}$ may have multiple execution modes, with each mode $m \in M_{i,j}$ corresponding to a specific worker requirement r_{ijm} and processing time d_{ijm}. Generally, an increase in the number of workers results in a reduction in the processing time for a given task. The total number of workers available is n, with no consideration given to differences in worker skill, and the assembly efficiency of the workers is assumed to be constant. The number of workers allocated to each workstation, as well as the start time and execution mode for each task, need to be determined to optimize a lexicographical bi-objective. The primary objective is to minimize the cycle time, while the secondary objective is to minimize the worker input.

The following assumptions are made to model the problem:

(1) Each task has a non-preemption duration.

(2) The execution mode cannot be altered once the task has commenced.

(3) Workers cannot be transferred between workstations during the assembly cycle.

3.2. Mathematical formulation

To rigorously describe the problem and solve it using a commercial solver, a lexicographic integer programming model is presented as follows. The parameters and decision variables are demonstrated in Table 2.

$$\text{Primary minimize } C \tag{1}$$

$$\text{Secondary minimize } \sum_{i=1}^{k} n_i \tag{2}$$

subject to

$$C \geqslant t_{i,j} + \sum_{m=1}^{|M_{i,j}|} d_{ijm} x_{ijm}, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i \tag{3}$$

$$\sum_{m \in M_{i,j}} x_{ijm} = 1, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i \tag{4}$$

Table 2
The definition of the parameters and decision variables.

Categories	Symbol	Meaning
Parameters	C	The cycle time.
	$Pre_{i,j}$	The predecessors of task $J_{i,j}$.
	$A_{i,j}$	The tasks being executed at time $t_{i,j}$.
Decision variables	$t_{i,j}$	The start time of task $J_{i,j}$.
	x_{ijm}	The 0-1 variable that decides whether the execution mode of task $J_{i,j}$ is m.
	n_i	The number of workers allocated to workstation w_i.

$$\sum_{i=1}^{k} n_i \leqslant n \tag{5}$$

$$t_{i,j} \geqslant t_{i,q} + \sum_{m=1}^{|M_{i,q}|} d_{iqm} x_{iqm}, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i, \forall q \in Pre_{i,j} \tag{6}$$

$$\sum_{q \in A_{i,j}} \sum_{m=1}^{|M_{i,q}|} r_{iqm} x_{iqm} \leqslant n_i, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i \tag{7}$$

$$t_{i,q} \leqslant t_{i,j} < t_{i,q} + \sum_{m=1}^{|M_{i,q}|} d_{iqm} x_{iqm}, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i, \forall q \in A_{i,j} \tag{8}$$

$$t_{i,j} \geqslant 0, \text{integer}, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i \tag{9}$$

$$x_{ijm} \in \{0, 1\}, \forall 1 \leqslant i \leqslant k, \forall 1 \leqslant j \leqslant m_i, \forall m \in M_{i,j} \tag{10}$$

$$n_i > 0, \text{integer}, \forall 1 \leqslant i \leqslant k \tag{11}$$

$$C > 0, \text{integer} \tag{12}$$

The objective function (1) aims to minimize the cycle time, while the objective function (2) seeks to minimize the total resource consumption. As the two objectives are both integers, to reflect the idea of lexicographic optimization in the solver, these objectives are consolidated into a single weighted objective function.

$$\text{minimize } C + \sum_{i=1}^{k} n_i / n \tag{13}$$

By incorporating a weight of $1/n$, we transform the lexicographical bi-objective into a single objective function with weights, as shown in (13). The proof of equivalence between the lexicographic optimization objectives (1)–(2) and the weighted single objective (13) is provided in Appendix. Constraints (3) guarantee that all tasks are completed within the cycle time. Constraints (4) ensure that only one mode is selected for each task. Constraints (5) indicate that the sum of workers allocated to each workstation does not exceed the total number of available workers. Constraints (6) stipulate that the precedence constraints are respected. Constraints (7) imply that at any given moment, the demand for workers at each workstation does not surpass the total number of its available workers. Constraints (8) demonstrate the conditions satisfied by two tasks executed in parallel. Constraints (9)–(12) define the properties of the decision variables.

4. Methods

To address large-scale instances of the proposed problem, where commercial solvers fail to provide feasible solutions within a reasonable timeframe, we propose an RReGA algorithm. Initially, a procedure is implemented to estimate a resource-makespan mapping for each workstation. Subsequently, the optimal resource allocation and cycle time are determined based on these mappings, utilizing a method that samples critical mapping points to expedite computation. Following this, a genetic operation is employed to search for scheduling solutions. Upon identifying an improved scheduling solution for a given resource allocation, a mapping update strategy is employed to refine the mappings. After a predefined evolution cycle, resource reallocation is executed to adjust and optimize the resource allocation. This resource reallocation method encompasses two key strategies: resource transfer and resource recovery. The resource transfer strategy redistributes workers from lower-makespan workstations to bottlenecked ones, thereby reducing cycle time and achieving balance across workstations. Conversely, the resource recovery strategy focuses on reclaiming excess resources, thereby minimizing resource investment without compromising scheduling efficiency. Detailed descriptions of each algorithmic step are provided in subsequent sections.

4.1. Resource-makespan mappings

The function $f_i : \mathbb{N}^+ \rightarrow \mathbb{R}^+$ maps resource investments to the makespan of the assembly task set J_i. These mappings $\{f_1, f_2, \ldots, f_k\}$ facilitate obtaining optimal resource allocation and cycle time. However, due to the significant computational demands, achieving the optimal makespan for all resource investments is impractical. Consequently, dispatch rule-based methods are commonly employed to quickly obtain feasible solutions. Nevertheless, a single scheduling rule typically is not universally suitable for all problems. To address this issue, we employ HDRs and a serial scheduling generation scheme (SSGS) to expedite the estimation of these mappings. It is crucial to acknowledge that such estimations introduce deviations from the exact mappings, which potentially misleads the optimality of resource allocation and cycle time. Therefore, the resource reallocation and mappings update strategy are indispensable.

4.1.1. Mappings estimation

The resource interval, namely the minimum resource investment (R_i^{min}) and maximum resource investment (R_i^{max}) for workstation w_i, must be determined before estimation. The R_i^{min} denotes the minimal resources required by each workstation to complete its assembly tasks, equating to the maximum resources needed for a single task in the assembly task set, executed in the mode demanding the least resources, as detailed in Eq. (14).

$$R_i^{min} = \max \left\{ \min \left\{ r_{ijm} \right\}_{m \in M_{i,j}} \right\}_{1 \leqslant j \leqslant m_i}, 1 \leqslant i \leqslant k \tag{14}$$

For a given assembly task set J_i, the maximum resource requirement, denoted as R_i^1, depends on the topological structure and execution modes of J_i. The makespan does not decrease when resource investment exceeds R_i^1. Let $\{S_{i,p}\}_{p=1}^{l}$ represent all subsets of J_i, where l is the total number of subsets, and each $S_{i,p}$ contains tasks that can be executed in parallel. Assume all tasks in $S_{i,p}$ are executed in the mode requiring the most resources, the resource requirement for each $S_{i,p}$, denoted as $R_{i,p}$, is calculated as shown in Eq. (15). Moreover, to ensure all workstations can complete their tasks, we require $\sum_{i=1}^{k} R_i^{min} \leqslant n$ and the number of worker allocated to workstation w_i no less than R_i^{min}. Thus, the maximum resource allocated to workstation w_i, denoted as R_i^2, is obtained as shown in Eq. (16).

$$R_i^1 = \max \left\{ R_{i,p} \right\}_{p=1}^{l}, 1 \leqslant i \leqslant k \tag{15}$$

$$R_i^2 = n - \sum_{q \in \{1,\ldots,k\} \setminus i} R_q^{min}, 1 \leqslant i \leqslant k \tag{16}$$

Then the R_i^{max} can be calculated as follows.

$$R_i^{max} = \min \left\{ R_i^1, R_i^2 \right\}, 1 \leqslant i \leqslant k \tag{17}$$

Given that the computation of R_i^1 is NP-hard [39], R_i^2 is utilized to estimate R_i^{max}. While this approach may entail an additional computation

Table 3
The dispatch rules of task selection and mode selection.

Role	DRs notation	Definition
Task selection	SPT	$\arg\min_{1\leqslant j\leqslant m_i}\left\{\min\left\{d_{ijm}\right\}_{m\in M_{i,j},r_{ijm}\leqslant n_i}\right\}$.
	LPT	$\arg\max_{1\leqslant j\leqslant m_i}\left\{\min\left\{d_{ijm}\right\}_{m\in M_{i,j},r_{ijm}\leqslant n_i}\right\}$.
	MST	$\arg\max_{1\leqslant j\leqslant m_i}\left\lvert Suc_{i,j}\right\rvert$.
Mode selection	MPT	$\arg\min_{m\in M_{i,j},r_{ijm}\leqslant n_i} d_{ijm}$.
	MRU	$\arg\max_{m\in M_{i,j},r_{ijm}\leqslant n_i} r_{ijm}/d_{ijm}$.

burden if $R_i^2 > R_i^1$, in most scenarios of the AFAL, this approximation remains feasible due to insufficient worker resources.

Our approach considers both task and mode selection dispatch rules. Task selection involves widely used rules such as shortest processing time (SPT), longest processing time (LPT), and maximum successor tasks (MST). The rules of mode selection involves minimum processing time (MPT) and maximum resource utilization (MRU). Details are summarized in Table 3, where $Suc_{i,j}$ represents the set of successor tasks of task $J_{i,j}$.

Each dispatch rule integrates a specific task selection rule with a mode selection rule, resulting in six unique HDRs. These HDRs generate task and mode lists for assembly tasks at each workstation, facilitating scheduling scheme decoding through SSGS to determine the makespan, with the minimum value viewed as the estimated mapping.

4.1.2. Optimal resource allocation based on the mappings

Based on these mappings, the resource allocation problem is formulated as follows.

$$\text{minimize } C = \max\left\{f_1\left(n_1\right), f_2\left(n_2\right), \ldots, f_k\left(n_k\right)\right\} \tag{18}$$

subject to

$$\sum_{i=1}^{k} n_i \leqslant n \tag{19}$$

$$n_i \geqslant R_i^{min} \tag{20}$$

The objective (18) is to minimize the cycle time, and $f_i(n_i)$ indicates the makespan of workstation w_i. The optimal solution, namely the optimal resource allocation can be obtained by traversing all allocation schemes but is computationally expensive. A special characteristic of these mappings significantly reduces unnecessary iterations, thereby enhancing computational efficiency. For a given set of assembly tasks, the makespan does not immediately decrease with an increase in resource investment. Instead, it exhibits periods where the curve remains constant. This phenomenon creates a distinct step-like pattern in the mapping curve, resembling a ladder, as depicted in Fig. 1.

We define the onset of each constant makespan period as a critical point, marking the minimum resource necessary to achieve a specific makespan, illustrated by the blue hexagon labeled with the makespan value in Fig. 1. By sampling these critical points, the solution to the problem (18) can be efficiently derived, which requires evaluation of just a few resource allocation schemes.

4.2. Genetic algorithm

The scheduling of assembly tasks on each workstation can be conceptualized as the MRCPSP, where each workstation operates under a given resource allocation scheme. Utilizing the RD policy, the scheduling challenges at individual workstations become independent post-allocation, enabling concurrent execution of GA to explore solutions.

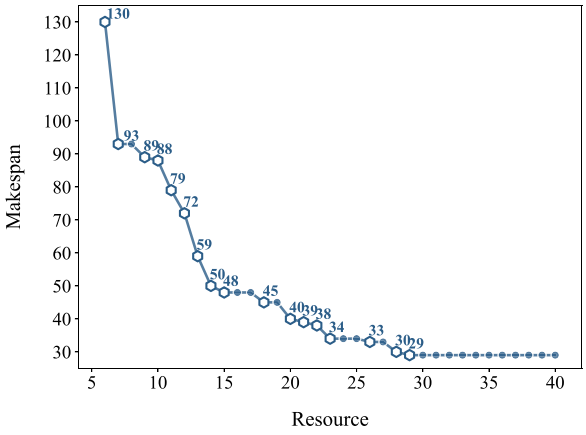


Fig. 1. The resource-makespan mapping.

Table 4
A simple instance of workstation assembly tasks with a total resources of 10.

Task	Modes	Predecessors
1	(7,4), (1,14)	–
2	(5,7), (2,13)	–
3	(7,7), (4,12), (3,13)	1
4	(7,5), (6,7), (5,10)	2, 3
5	(7,8), (4,11), (2,15)	4
6	(7,3), (6,4), (3,5)	4
7	(7,2), (6,9), (5,10)	5, 6

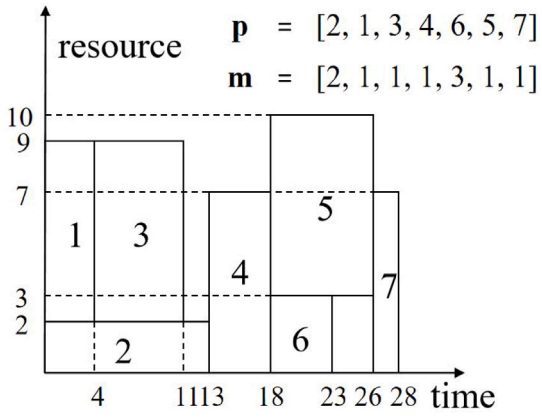


Fig. 2. The gantt chart of a simple instance.

4.2.1. Solution encoding and decoding

Each individual's chromosome comprises two components $(\mathbf{p}, \mathbf{m})$. Here, $\mathbf{p} = (p_1, p_2, \ldots, p_{m_i})$ represents a task permutation, with p_j denoting the task index at position j. The second part, $\mathbf{m} = (m_1, m_2, \ldots, m_{m_i})$, signifies task execution mode selections, where each element specifies the required number of workers and processing time for the corresponding task. To ensure solution feasibility, permutation $\mathbf{p}$ must adhere to precedence constraints, ensuring each task follows its predecessors. Meanwhile, mode selection in $\mathbf{m}$ must satisfy resource constraints, ensuring that the resource demand for each task does not surpass available resources.

Upon determining a solution represented by task list $\mathbf{p}$ and mode list $\mathbf{m}$, decoding proceeds via SSGS to determine the start processing time and completion time of each task, culminating in the makespan. A simplified instance is illustrated in Table 4, with corresponding encoding (task list $\mathbf{p}$ and mode list $\mathbf{m}$) and decoding (Gantt chart) outcomes depicted in Fig. 2.

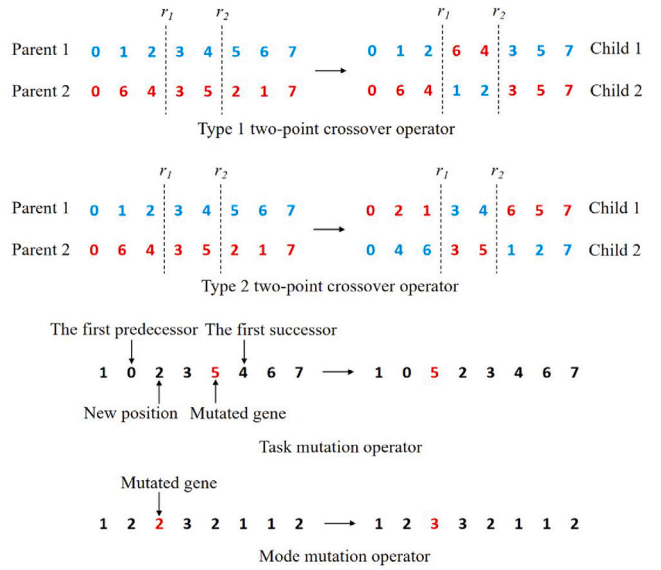


Fig. 3. Crossover and mutation operators.

4.2.2. Initial population and parent selection

To begin with, an initial population POP_0 of size N is generated. Chromosomes are initialized randomly: for task list **p**, tasks are selected only after all preceding tasks have been scheduled, ensuring feasibility. For mode list **m**, feasible modes for a task are determined based on available resources, with one mode selected randomly. This method guarantees that all solutions are inherently feasible, obviating the need for preprocessing before population evolution.

To maintain offspring diversity, each individual in POP_0 serves as a parent, creating offspring through random division into two equally sized sets (P_1, P_2). Pairs of parents are sequentially sampled from these sets without replacement, respectively, to generate $N/2$ pairs, each producing two children.

4.2.3. Crossover and mutation

The crossover operators are applied with probability p_c only on the task list **p** to generate children. We use two types of "two-point" crossover operators, known to maintain solution feasibility (refer to [40] for detailed operator descriptions). While crossover does not directly manipulate mode list **m**, the coupling of **p** and **m** ensures inherited gene consistency across offspring.

The mutation operators are deployed on both **p** and **m** with probability p_m. Mutation in **p** involves altering the position of a gene p_i, ensuring the modified individual remains feasible. This process involves selecting a task p_i and repositioning it within permissible bounds, adjusting adjacent genes accordingly. Mutation in **m** entails selecting a task p_i and replacing its original mode m_i with a randomly chosen feasible mode m_i'. Fig. 3 details the implementation of these operators.

4.2.4. Fitness computation and population update

Evaluation of individuals precedes offspring retention and elimination. Fitness value is determined by the makespan obtained post-decoding, as shown in Eq. (21). Lower values denote superior quality.

$$fit = C_{max} = C_i\left(\mathbf{p}, \mathbf{m}, n_i\right) \tag{21}$$

Following offspring generation from parental POP_i, the population size increases to $2N$. To enhance overall population quality while preserving diversity, offspring are stratified by fitness into tiers, with the top $0.25N$ individuals retained. The remaining individuals are split into two sub-populations in a $2:1$ ratio, from which $0.5N$ and $0.25N$ individuals are selected to form the subsequent population POP_{i+1}.

4.2.5. Mapping update strategy

For a given set of assembly tasks J_i with resource n_i, the non-increasing mapping of makespan to resource investment ensures the ability to refine its mapping f_i once a better makespan C_{max} is obtained during the GA execution.

$$f_i(x) = \begin{cases} C_{max}, \text{if } x \geqslant n_i \text{ and } f_i(x) > C_{max} \\ f_i(x), \text{otherwise} \end{cases} \tag{22}$$

As described in function (22), this strategy replaces the previous makespan with the newly determined optimal makespan for resource investments exceeding n_i. With the mappings update strategy, the mapping of each workstation will be closer to the globally optimal value, potentially yielding a closer-to-optimal resource allocation and cycle time to the problem (18). This adjustment guides the subsequent resource reallocation.

4.3. Resource reallocation

The significant deviation in mapping estimation attributed to the hybrid scheduling rule implies that the effectiveness of the initial resource allocation from solving the problem (18) is not assured. As the GA optimizes the makespans of workstations, an inadequate resource allocation scheme can exacerbate the imbalance in workstation makespans and hinder further optimization of cycle time due to insufficient resources at the bottleneck workstation.

To enhance resource allocation and optimize the objective function, we propose an efficient resource reallocation method comprising a resource transfer strategy and a resource recovery strategy. The resource transfer strategy involves augmenting resources at bottleneck workstations by reallocating excess resources from over-resourced workstations. It is worth mentioned that the assumption (3) in Section 3.1 applies solely to the final production schedule. The resource reallocation operations during the algorithm's intermediate phase aim to optimize solutions for deriving the final production schedule, without modifying the actual final production instructions. This redistribution temporarily increases makespans at non-bottleneck workstations, achieving balanced workstation cycle times. Subsequent to resource transfers, the cycle time optimization concludes, and a resource recovery policy retrieves surplus resources from each workstation to manage resource cost. This recovery strategy ensures that each workstation maintains a makespan not exceeding the cycle time with minimal resource investment.

4.3.1. Resource transfer strategy

Prior to resource transfer, identification of bottleneck workstations and non-bottleneck workstations is essential.

Definition 1 (*BWS and NBWS*)**.** For a given workstation w_i, if $f_i(n_i) \geqslant C$, it is a bottlenext workstation (BWS); conversely, it is a non-bottleneck workstation (NBWS).

Definition 2 (*Resource Pool*)**.** A shared repository for resources contributed by the NBWS.

Definition 3 (*Resource Buffer*)**.** Exclusive temporary resource storage for each workstation.

A principle is constructed for each workstation to follow according to its type and resource buffer status.

Type (1) The workstation is an NBWS and its resource buffer is empty.

Let r_i^{out} denotes the number of resources transferred by workstation w_i in a single transfer. Deposit r_i^{out} resources into its resource buffer. This workstation possesses the potential to transfer resources, and an empty resource buffer indicates that the workstation either received

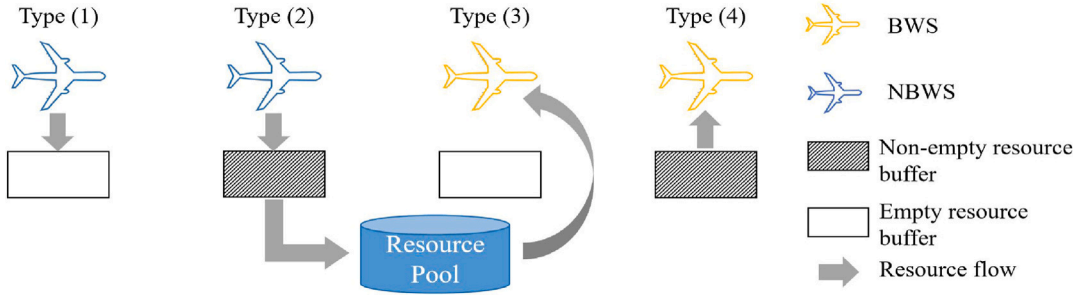


Fig. 4. Four types of workstations in resource transfer.

resources in the preceding round of resource transfer or this is the initial round of resource transfer. In instances where such an NBWS cannot ensure that it will not deteriorate into a bottleneck workstation post resource transfer, it temporarily deposits a portion of the resources into the resource buffer to prevent acquisition by other workstations.

Type (2) The workstation is an NBWS and its resource buffer is non-empty.

Transfer all resources from its resource buffer to the resource pool, and subsequently deposit r_i^{out} resources into the resource buffer. The presence of resources in the buffer indicates that the workstation has already contributed resources in the previous round and retains the potential to transfer additional resources in the current round. Consequently, the resources allocated in the previous round can be entirely redirected to the resource pool for utilization by the bottleneck workstations. Furthermore, the resources transferred out in the current round can be deposited into the resource buffer.

Type (3) The workstation is a BWS and its resource buffer is empty.

Allocate all the resources from the resource pool to this workstation. If the ties occur, the workstation with the smallest index receives all the resources. The emptiness of the resource buffer indicates that the workstation has exhausted its reserve resources. However, its status as a bottleneck workstation necessitates the acquisition of additional resources. Consequently, it obtains resources from the resource pool that have been entirely released by the other workstations.

Type (4) The workstation is a BWS and its resource buffer is non-empty.

Retrieve all resources from the resource buffer. The non-empty buffer indicates that the workstation contributed resources in the preceding round, and it is this transfer of resources that transformed it into a BWS. Therefore, it is imperative to retrieve the resources stored in the resource buffer promptly.

Eq. (23) shows the formulation of r_i^{out}, where r_i^{red} denotes the redundant resources of workstation w_i calculated based on its mapping and the optimal cycle time C. r_i^{red} is formulated by Eq. (24), where x_1 represents the minimum resource investment ($(x_1, f_i(x_1))$ is a critical point of the mapping f_i) for workstation w_i that makes its makespan equals to $f_i(x_2)$. Here, $f_i(x_2)$ finds a resource input x_2 that makes the workstation's makespan not exceed, but as close as possible to the cycle time.

$$r_i^{out} = \max\left\{r_i^{red}, 1\right\} \tag{23}$$

$$r_i^{red} = n_i - \arg\min_{x_1}\left(f_i\left(x_1\right) = \min_{f_i(x_2), f_i(x_2) \leqslant C}\left(C - f_i\left(x_2\right)\right)\right) \tag{24}$$

It is evident that only workstations in type (3) utilize resources from the resource pool. Suppose such workstations are absent in a given round of resource transfer. In that case, it results in a continuous accumulation of resources in the resource pool and a corresponding decrease in the resources circulating among workstations. To mitigate this issue, when no type (3) workstations exist, due to the inevitability of the existence of BWS, type (4) workstations must exist at that time, and all the resources in the resource pool are equally distributed to all type (4) workstations. This strategy guarantees that the resource pool is emptied after each round of resource transfer, thereby preventing the resource backlog within the resource pool. The pseudo-code and details of the resource transfer strategy are illustrated in Algorithm 1, and Fig. 4 describes the mechanism of this strategy.

Algorithm 1 The resource transfer strategy

Input: resource allocation $\{n_1, n_2, ..., n_k\}$, resource-makespan mappings $\{f_1, f_2, ..., f_k\}$, resource buffers $\{B_1, B_2, ..., B_k\}$.
Output: updated resource allocation $\{n_1', n_2', ..., n_k'\}$, updated resource buffers $\{B_1', B_2', ..., B_k'\}$.

Set the resource pool $P \leftarrow 0$ and $flag \leftarrow$ **true**;
Calculate the optimal cycle time C based on $\{f_1, f_2, ..., f_k\}$;
Determine the bottleneck workstations and non-bottleneck workstations;
for i from 1 to k **do**
  if w_i is an NBWS **then**
    if $B_i = 0$ **then**
      Calculate the r_i^{out};
      $B_i' \leftarrow r_i^{out}$, $n_i' \leftarrow n_i - r_i^{out}$;
    else
      $P \leftarrow P + B_i$;
      Calculate the r_i^{out};
      $B_i' \leftarrow r_i^{out}$, $n_i' \leftarrow n_i - r_i^{out}$;
    end if
  end if
end for
for i from 1 to k **do**
  if w_i is a BWS **then**
    if $B_i = 0$ **then**
      $n_i' \leftarrow n_i + P$, $P \leftarrow 0$, $flag \leftarrow$ **false**;
    else
      $n_i' \leftarrow n_i + B_i$, $B_i' \leftarrow 0$;
    end if
  end if
end for
if $flag$ is **true then**
  Distribute P equally to all BWSs;
  $P \leftarrow 0$;
end if
return $\{n_1', n_2', ..., n_k'\}$, $\{B_1', B_2', ..., B_k'\}$.

In the process of resource transfer, all workers are engaged in the assembly task without consideration for minimizing resource cost. Each workstation has a resource investment upper bound R_i^1 based on its assembly tasks topology, as shown in Eq. (15), and any resource investments exceeding this bound are futile. When a workstation reaches its upper bound but remains a bottleneck workstation, the resource transfer policy continues to allocate additional resources to it, particularly when the total resources are rich. Although this practice does not impact the optimization of the cycle time, it is detrimental to controlling the workstation's resource cost and precludes the exploration

of scheduling solutions under low-resource conditions. Therefore, the subsequent resource recovery strategy is designed to cope with these issues.

4.3.2. Resource recovery strategy

Upon completing resource transfers and achieving the optimal cycle time C, the temporarily minimum resource requirements for workstations are determined, as shown in Eq. (24). At this stage, we have derived the resource allocation solution $\{n_1, n_2, \dots, n_k\}$ for the primary-objective (1) optimization. After cycle time optimization, further resource input reduction is pursued to optimize the secondary objective function.

Throughout the resource transfer process, certain workstations may consistently act as bottleneck workstations to acquire new resources. Consequently, their genetic algorithms are always executed in a resource-rich context. However, in scenarios with limited resources, these workstations lack sufficient opportunities to update the mapping of this interval. In fact, they may have the potential to excel in achieving a makespan no greater than the cycle time C with reduced resource input. Based on such a core idea, the resource recovery strategy aims to enable all workstations to explore their potential to complete all assembly tasks within C with fewer resources than currently allocated.

Specifically, after determining the best-so-far resource input for each workstation by solving (24), the resource recovery strategy is carried out by reducing one-unit resource input at each workstation. At this point, the makespans of all workstations will exceed the cycle time C. The workstations then continue to execute the GA in an attempt to find a more optimal scheduling solution to reduce the makespan no greater than C. Once achieved, the resource input for the workstation will be reduced by one unit again, and the optimization of the scheduling solution continues. Otherwise, if such a solution cannot be found during this process, then keep the previous resource input and scheduling solution. This strategy is repeated until the iteration stopping condition is satisfied, as shown in Condition 4 of Fig. 5, resulting in the final optimal solution to the problem (13).

4.4. RReGA

Executing the GA afresh with each resource allocation update would impose a significant computational burden and severely affect the efficiency. In fact, resource adjustments in our context can be treated as changes in the environment of the GA population. The existing GA populations can still be employed to represent scheduling solutions after resource reallocation. It suffices to reassess the fitness of individuals in this new environment, no need to restart the GA from scratch, ensuring continuity in evolution. In practice, the amount of resource adjustment per iteration is minimal, and as the evolution progresses, both resource allocation and the makespan of each workstation stabilize. The degree of environmental change becomes increasingly subtle, maintaining the population's adaptability to the new environment.

For the resource transfer strategy, it is necessary to reserve sufficient time for the evolution of the populations at each workstation after each resource adjustment. This ensures that the populations can identify the best-adapted individuals to the new environment, thereby accurately reflecting the performances of the workstations post-adjustment. Conversely, the resource recovery strategy does not have a fixed execution period. Adjustments can be made once the population's performance meets expectations. The flowchart of the RReGA is illustrated in Fig. 5.

T^{rt} denotes the execution period for the resource transfer strategy, implying that the population undergoes T^{rt} evolutionary iterations. During a resource transfer, if no workstation contributes resources to the resource pool, it indicates that all non-bottleneck workstations cannot maintain their status if resources are redistributed to the bottleneck workstation. This situation suggests that no resource transfers should be made at that moment. If this condition persists for s_1 consecutive times without any optimization in cycle time C by the bottleneck workstation (condition 2 in Fig. 5), it indicates that the workstation can no longer decrease its makespan (i.e., the assembly cycle) through either resource acquisition from other workstations or optimization of its task list $\mathbf{p}$ and mode list $\mathbf{m}$. Thus, the resource transfer strategy ceases and transitions to the resource recovery phase. Condition 3 is triggered when a workstation satisfies the constraint of cycle time C by optimizing its scheduling scheme after the resources have been recovered. This means that the previous round of resource recovery is feasible and further resource recovery from the workstation can be attempted, i.e., the start of a new round of resource recovery. The termination condition for the resource recovery strategy (Condition 4 in Fig. 5) is triggered when no resources are recovered over s_2 consecutive iterations. This signifies that all workstations have reached a minimal resource requirements while maintaining makespans that do not exceed the cycle time C. The RReGA algorithm concludes when resource recovery stops or the maximum number of evolutionary iterations is reached.

In cases where total worker resources are extremely scarce or the topology of a specific assembly task set is peculiar (e.g., tasks are performed serially), the resources allocated to other workstations may fall into a critical range. Decoding solutions with the original population in the new environment may fail in such scenarios. As depicted in Eq. (14), R_i^{min} has been defined in the previous text as the minimum resource requirement for a workstation to complete all assembly tasks. The maximum resource requirement for a single task in assembly task set J_i is denoted as R_i^{sm}, as shown in Eq. (25).

$$R_i^{sm} = \max\left\{\max\left\{r_{ijm}\right\}_{m\in M_{i,j}}\right\}_{1\leqslant j\leqslant m_i}, 1 \leqslant i \leqslant k \tag{25}$$

When resource investment falls within the interval $[R_i^{min}, R_i^{sm}]$, some individuals may fail to decode due to insufficient resources for specific task requirements, necessitating the reinitialization of the workstation's population to ensure feasibility and continue the algorithm's subsequent iterations. For instance, as illustrated in Table 4, with $R_i^{min} = 5$ and $R_i^{sm} = 7$, the individual in Fig. 2 thus requires at least 7 resources (workers), while post resource transfer the resource input is 5 or 6, the individual will be unable to adapt to the new environment. Since this interval is derived from the resource requirements of individual tasks, the search spaces in terms of this interval are highly constrained, resulting in significantly smaller solution spaces, which leads to high search efficiency. Consequently, even without inheriting the previous populations, the impact on efficiency is subtle.

5. Experiments and discussions

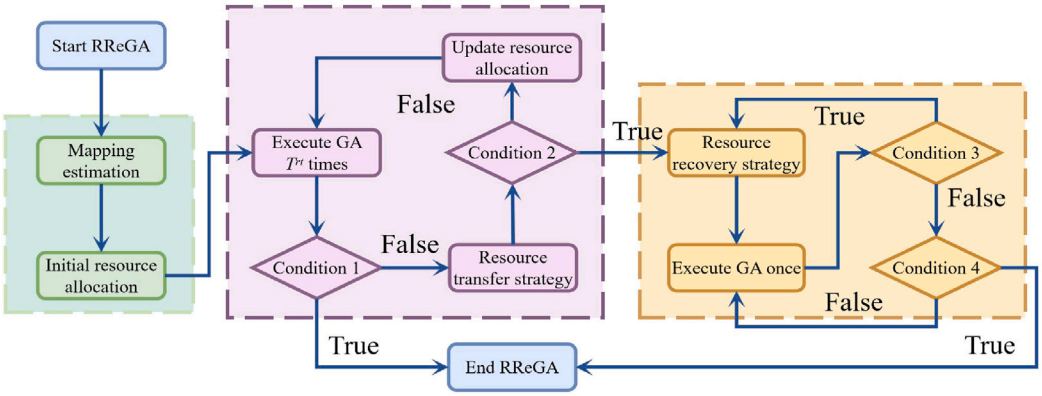
Numerous experiments have been conducted to validate the superiority of the proposed method. All the approaches were implemented using Python 3.10 on a computer with an Intel Core i5-10210U 1.60 GHz processor and 16 GB of RAM. The mathematical model was solved utilizing GUROBI 10.0.

5.1. Preparation for experiments

5.1.1. Datasets generation

Regarding the parameters associated with dataset generation, we refer to the values deployed in [33]. These parameters include the number of tasks per workstation ($|J_i|$), the maximum number of feasible execution modes for each task ($\max_{1\leqslant i\leqslant k, 1\leqslant j\leqslant m_i}|M_{i,j}|$), task processing time (d_{ijm}) and worker requirement (r_{ijm}). The value ranges for these parameters are detailed in Table 5.

Utilizing the parameter configurations specified in Table 5, we generated six distinct datasets, each comprising 50 instances, resulting in a total of 300 assembly task instances. The number of tasks of a single instance in each set is 5, 7, 10, 15, 20, and 30, respectively.



- Condition 1 : The maximum number of iterations *Iter* is reached.
- Condition 2 : BWS is not resourced and the cycle time C is not decreasing for s_1 consecutive times.
- Condition 3 : The makespan decreases to the cycle time C.
- Condition 4 : No resources are reclaimed for s_2 consecutive iterations or the maximum number of iterations *Iter* is reached.

Fig. 5. Flowchart of RReGA.

Table 5
Parameters setting of datasets generation.

Parameter	Value
$\lvert J_i \rvert$	{5, 7, 10, 15, 20, 30}
$\max \lvert M_{i,j} \rvert$	5
d_{ijm}	[1, 15]
r_{ijm}	[1, 7]

Table 6
Parameters setting.

Instance	Workers	Population size
5	50	30
7	60	30
10	70	40
15	80	40
20	90	50
30	100	50

5.1.2. Experimental design

For a given instance, the optimal solution and objective value of problem (13) vary significantly in terms of the total number of workers. The situations of worker input can be categorized into two types: sufficient worker input and insufficient worker input. The former indicates that, with a reasonable resource allocation scheme, the worker input of the bottleneck workstation w_i can exceed its maximum resource requirement R_i^1, whereas the latter cannot achieve it. Therefore, in the following experiments, each instance will be tested under both scenarios. Sufficient worker input for instances of various sizes can be determined through preliminary experiments (as shown in Table 6), while insufficient input vary according to the selected instance. The value of insufficient worker input in each experiment will be randomly chosen from the interval $\left[\sum_{i=1}^{k} R_i^{min}, R^{used}\right]$, where R^{used} represents the resource consumption corresponding to the solution obtained with sufficient worker input.

Instances are selected randomly from datasets with task numbers 5, 7, 10, 15, 20, and 30, forming 20 experiments, respectively. Each experiment containing 5 ($k = 5$, the number of workstations) instances with the same number of tasks. Each experiment will be conducted with both sufficient and insufficient worker input, totaling 240 experiments. Before the formal experiments, a series of preliminary experiments are conducted to tune parameter values, balancing solution efficiency and quality. The crossover probability (p_c) and mutation probability (p_m) of the RReGA are set to 0.9 and 0.1, respectively. The population size (N) and sufficient resource input are shown in Table 6, and the maximum number of iterations (*Iter*) is 200. The resource transfer period (T^{rt}) refers to 5 and 10 iterations under sufficient and insufficient worker input, respectively. s_1 and s_2 are set to 5 and 10, respectively.

Table 7
Time consumption for mappings estimation.

Instance	5	7	10	15	20	30
CPU time-0 (s)	0.53	0.96	1.38	2.90	5.66	12.64
CPU time-1 (s)	0.14	0.24	0.42	0.64	1.03	1.47

5.2. Performance of mappings estimation

The resource-makespan mappings serve three principal functions: providing an initial resource allocation, facilitating resource transfers, and determining the optimal resource allocation scheme. The precision and efficiency of these mappings are critical to the overall performance of the RReGA algorithm. To assess this, we investigate the time expenditure associated with mapping estimation across various instances and perform comparative experiments to evaluate the estimation deviations relative to two alternative heuristic methods for initial resource allocation.

Based on Eq. (14) and (17), the quantity of mapping estimations is dependent upon the availability of both workers and workstations. In scenarios where worker resources are inadequate, the number of mapping points requiring estimation diminishes. We conducted empirical investigations involving task instances of 5, 10, 15, 20, and 30, each with 5 workstations, and measured the time consumption for mapping estimation under two distinct levels of resource allocation.

Table 7 delineates the results from these experiments. In this table, "CPU time-0" denotes the time consumption with adequate resource allocation, while "CPU time-1" represents the time consumption under conditions of insufficient resource allocation. The data reveal that the time expenditure escalates markedly with the increase in task number, and "CPU time-1" exhibits a substantial reduction relative to "CPU time-0". This result supports our hypothesis that less resource availability implicates a decrease in the number of required estimations. Additionally, the effectiveness of the HDRs-based mapping estimation method is evidenced by its performance, which achieves an average estimation time of 12.64 s even in the intricate case of 30 tasks and 5 workstations with ample worker resources. This duration is further

Table 8
Deviations of different resource allocation estimation methods.

Instance	Average deviation		
	HDRs	S1	S2
5–0	**1.187**	2.333	2.291
5–1	**0.494**	1.101	1.153
7–0	**1.087**	3.573	3.345
7–1	**0.452**	0.998	0.938
10–0	**1.644**	2.998	3.091
10–1	**0.588**	0.965	0.962
15–0	**2.449**	3.873	3.868
15–1	**0.492**	0.943	0.881
20–0	**1.477**	3.839	3.787
20–1	**0.463**	0.738	0.784
30–0	**2.853**	3.736	3.662
30–1	**0.412**	0.481	0.523

significantly reduced under the same scale when faced with a shortage of workers.

The initial resource allocation significantly influences the convergence rate of the RReGA algorithm. A comprehensive evaluation with regard to the performance of the resource allocation scheme derived from the mappings is imperative. Given that GUROBI cannot provide the optimal solution to the problem (13) for large-scale instances within an acceptable time, we utilize the final resource allocation obtained by RReGA as a benchmark to calculate the estimated deviation of the initial resource allocation. Considering the optimal resource allocation $\sigma^* = \{n_1^*, n_2^*, \ldots, n_k^*\}$ and the initial resource allocation $\sigma^0 = \{n_1^0, n_2^0, \ldots, n_k^0\}$, the weighted estimation deviation is defined as follows.

$$DEV = \sum_{i=1}^{k} \frac{\left\| n_i^* - n_i^0 \right\|}{n_i^*} \tag{26}$$

Two statistical heuristic methods, S1 and S2, are employed to compare with the initial resource allocation estimation method presented in this paper. Method S1 allocates resources based on the number of tasks at each workstation, with the number of workers assigned to the workstation w_i calculated as shown in Eq. (27). Comparatively, Method S2 allocates resources based on the aggregate average resource requirements of all tasks at a workstation, as calculated in Eq. (28).

$$n_i = n \times \frac{|J_i|}{\sum_{l=1}^{k} |J_l|}, 1 \leqslant i \leqslant k \tag{27}$$

$$n_i = n \times \frac{\sum_{l=1}^{|J_i|} \left(\frac{\sum_{m \in M_{i,l}} r_{ilm}}{|M_{i,l}|} \right)}{\sum_{p=1}^{k} \sum_{l=1}^{|J_p|} \left(\frac{\sum_{m \in M_{p,l}} r_{plm}}{|M_{p,l}|} \right)}, 1 \leqslant i \leqslant k \tag{28}$$

Table 8 reports the average estimation deviations of three methods across 20 experiments conducted on various instances. Each instance is categorized with labels: "-0" for sufficient resources and "-1" for insufficient resources. As highlighted in the table, the mean deviations of the estimation method proposed in this paper are significantly lower across all instances compared to the two other statistical heuristic methods. This indicates a more rapid convergence to optimal allocation during subsequent resource transfers. Notably, the mean deviations improve markedly under conditions of insufficient resource input relative to sufficient resource input, a pattern consistently observed across all three methods. Further investigation reveals that this phenomenon arises because, under sufficient resource conditions, the optimal allocation strategy does not fully utilize all workers but rather eliminates redundant resources through the resource recovery strategy—a consideration not incorporated in the other methods. In contrast, with insufficient resources, nearly all workers are fully utilized, leading to reduced deviations.

To more intuitively illustrate the comparative results, error bar charts depicting deviations under two resource input conditions are presented in Fig. 6. The bars represent the mean estimation deviations, and the error bars indicate the standard deviation.

5.3. Performance of RReGA

To assess the efficacy of the RReGA algorithm, this paper compares its performance against the GUROBI solver and state-of-the-art methods using multiple metrics. Additionally, extensive ablation experiments are conducted to investigate the influence of resource transfer and recovery strategies on the RReGA algorithm. Finally, to verify the algorithm's effectiveness on unbalanced instances, an unbalanced case is simulated and addressed using the proposed algorithm.

5.3.1. Comparison with GUROBI

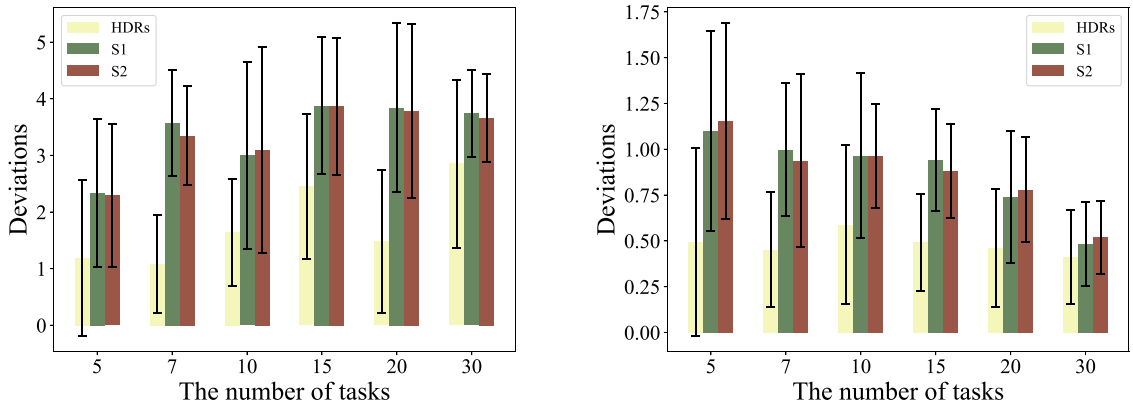
This section presents the experimental results for the GUROBI solver and the RReGA algorithm across various instance sizes, as detailed in Table 9. The maximum solution time for GUROBI is constrained to 3600 s. The average solution times for GUROBI and RReGA are denoted by t_G and t_R, respectively. The number of feasible solutions found by GUROBI in each set of 20 experiments is represented by #fea, while #opt indicates the number of optimal solutions. Define gap as below.

$$\mathrm{GAP}^{\mathrm{o}}\left(o_1, o_2\right) = \frac{o_1 - o_2}{\min\left\{o_1, o_2\right\}}, o \in \{C, R, O\} \tag{29}$$

The average gap in cycle time C between solutions from the two methods over the 20 experiments is denoted by GAP^C. GAP^R refers to the average gap in resource consumption for the same cycle times between the two methods. If the cycle times differ, the gap in cycle time C is recorded, otherwise the gap in resource consumption is noted. GAP^O indicates the average gap in the optimization objective value.

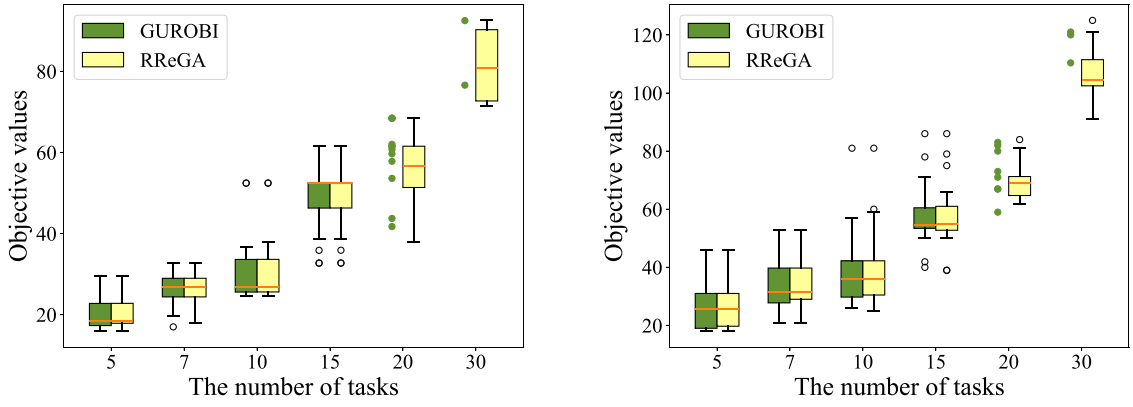
Efficiency stands out as one of the most notable advantages of the RReGA algorithm, particularly as the scale of instances increases. In cases with 30 tasks, the average computational time for RReGA is less than 300 s, while GUROBI struggles to provide a feasible solution within an hour. For all other instances, the proposed algorithm consistently demonstrates a time reduction by a factor compared to GUROBI. Furthermore, within instances of the same size, the computational time is higher when resource allocation is ample compared to when it is insufficient. This pattern is observed in the results from both methods, though the underlying reasons differ. For RReGA, the discrepancy arises from variations in the parameter T^{rt}. When resources are sufficient, multiple scheduling solutions can achieve the optimal cycle time C, allowing for more flexible resource allocation. Optimization occurs primarily during the subsequent resource recovery stage, necessitating a smaller T^{rt}. Conversely, with insufficient resources, a more precise and efficient resource allocation is required to achieve the optimal cycle time, resulting in higher computational cost during the resource transfer phase. In the case of GUROBI, which employs the branch-and-bound method and various heuristics to solve integer linear programming problem (13), the introduction of additional constraints when resources are limited leads to a greater number of branches. This complexity complicates the use of heuristics to find feasible solutions, thereby increasing the computational time.

As shown in Table 9, as the number of tasks increases, GUROBI increasingly struggles to obtain feasible and optimal solutions within one hour. For cases with more than 20 tasks, GUROBI becomes unsuitable. The results in Table 9 indicate that for small-scale instances, RReGA yields solutions close to the global optimum, with an average error of less than 2%. For large-scale instances, the solutions provided by the proposed algorithm are superior to those of GUROBI, with an average error being negative. The values of GAP^C and GAP^R in the table reveal that the factors causing gaps differ based on resource availability. When resources are ample, the gap between the solution provided by RReGA and GUROBI is primarily in terms of resource consumption (GAP^R). RReGA achieves the optimal cycle time C but



(a) sufficient resource (b) insufficient resource

Fig. 6. Estimation deviations of resource allocation.



(a) sufficient resource (b) insufficient resource

Fig. 7. Boxplots of objective values obtained by GUROBI and RReGA.

Table 9
Comparison of GUROBI and RReGA.

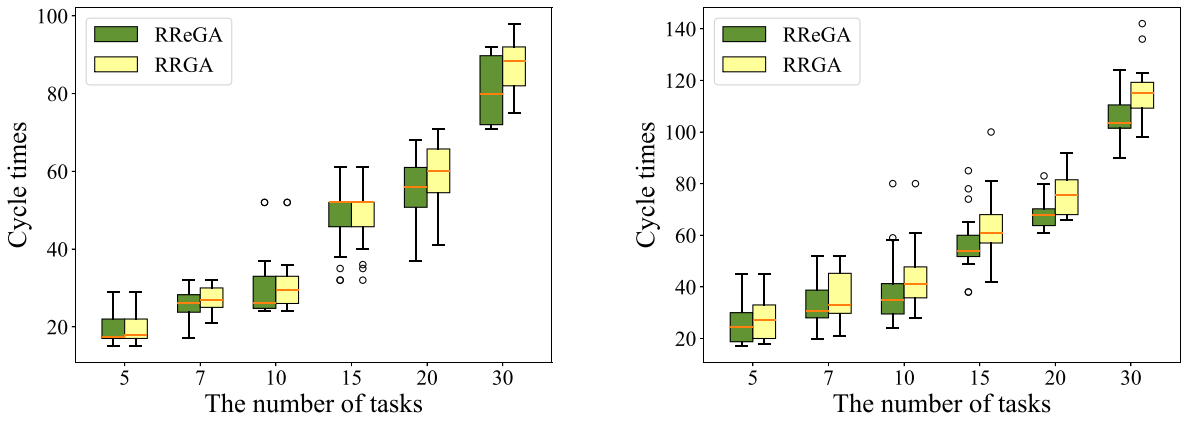
Instance	t_G (s)	#fea	#opt	t_R (s)	GAP^C (%)	GAP^R (%)	GAP^O (%)
5–0	4.15	20	20	7.06	0.667	0.251	0.640
5–1	7.70	20	20	8.76	0.111	0.147	0.256
7–0	194.32	20	19	11.92	0.313	0.122	0.288
7–1	215.01	20	19	12.65	0.524	0.000	0.521
10–0	413.15	20	18	30.06	0.301	2.417	0.361
10–1	1091.85	20	14	40.55	1.513	0.373	1.493
15–0	1816.65	20	13	91.51	−0.096	4.189	−0.051
15–1	3414.97	19	3	92.67	−0.068	0.663	−0.023
20–0	3530.72	13	1	120.12	−1.433	−0.132	−1.476
20–1	3594.55	8	1	126.97	−0.647	0.000	−0.640
30–0	3600.00	2	0	291.57	−0.352	−0.200	−0.353
30–1	3600.00	3	0	278.64	−0.015	0.000	−0.015

at a slightly higher resource consumption than the global optimum. Conversely, when resources are insufficient, the gap between them is more related to cycle time (GAP^C). These findings corroborate the earlier discussion regarding different resource input for the same instance: ample resources facilitate achieving the optimal cycle time but may lead to resource waste, whereas insufficient resources lead to full utilization but make optimal allocation difficult, resulting in cycle time deviations.

Fig. 7 presents an overview of the comparison results of the objective values for the two methods across different instances. The top and bottom lines of each boxplot represent the upper and lower quartiles of the objective function values, respectively. The central line in each box denotes the median of the objective values. The whiskers extending from the box indicate the maximum and minimum values. Outliers are marked with "o" and are excluded from the statistical summary. The green dots represent the objective values corresponding to a feasible solution that GUROBI is able to give in 3600 s on large-scale instances. Fig. 7 reveals that for small-scale instances with 5, 7, and 10 tasks, the boxplots for both methods are almost identical, whereas for large-scale instances with 15, 20, and 30 tasks, RReGA demonstrates superior optimization results.

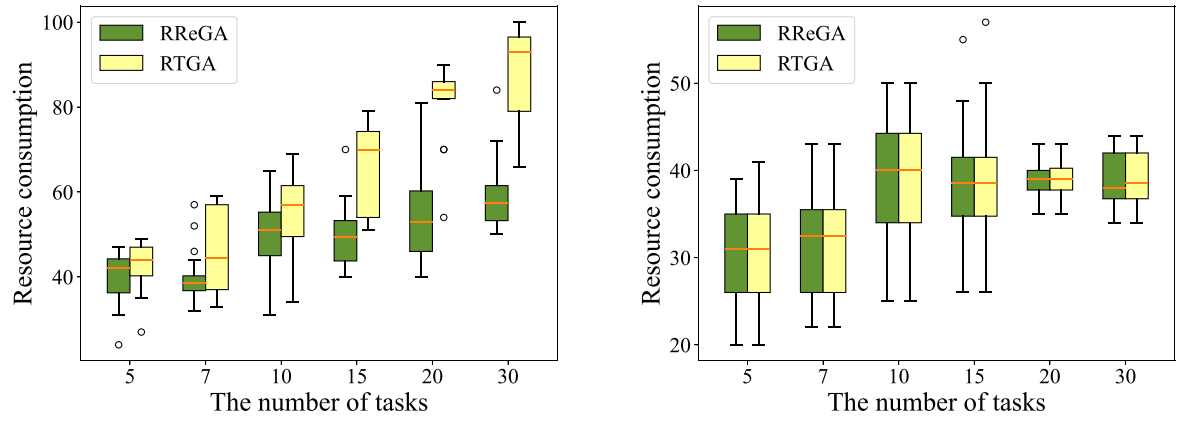
5.3.2. Ablation study of RReGA

To further investigate the impact of resource transfer and recovery strategies on the performance of the RReGA algorithm, ablation experiments on the same 240 instances were conducted. We evaluated



(a) sufficient resource (b) insufficient resource

Fig. 8. Boxplots of cycle time obtained by RReGA and RRGA.



(a) sufficient resource (b) insufficient resource

Fig. 9. Boxplots of resource consumption obtained by RReGA and RTGA.

the RReGA algorithm's performance after removing resource transfer and recovery components. The algorithm retaining only the resource transfer component was designated RTGA. RTGA transfers resources between workstations to optimize cycle time and balance the makespan, while ignoring the recovery of redundant resources. The algorithm with the resource transfer strategy removed is termed RRGA. RRGA performs makespan optimization and redundant resource recovery at each workstation after the initial resource allocation is obtained based on resource-makespan mappings. RRGA transitions from optimization of the cycle time to resource recovery stage when no superior offspring are found to reduce a workstation's makespan for 20 consecutive generations. RRGA and RTGA retain the other parameters of the RReGA algorithm.

Table 10 presents the results of various metrics for the three methods across 12 experimental sets. CPU time denotes the average duration for 20 experiments per set. RPD^C and RPD^R represent the average relative percentage deviation for the two objectives, namely cycle time and resource consumption. RTGA, retaining the resource transfer strategy, produces identical cycle time optimization results with those of RReGA, thus only RPD^R is compared. The primary difference between RRGA and the proposed algorithm lies in resource transfer, leading to significant variations in cycle time C optimization results. As a result, RPD^C serves as the evaluation metric. The definition for both RPD^C and RPD^R is given in Eq. (30).

$$RPD^o\left(o_1, o_2\right) = \frac{o_1 - \min\left\{o_1, o_2\right\}}{\min\left\{o_1, o_2\right\}}, o \in \{C, R\} \tag{30}$$

As shown in Table 10, the computational time for RReGA is slightly higher than for RTGA and RRGA across all instances. Fig. 10 illustrates the average processing time for four methods, including GUROBI, across different instance sizes. It is observed that the time consumption of GUROBI significantly exceeds that of the other methods. It is worth noting that in scenarios with insufficient resource input, RReGA and RTGA exhibit nearly identical processing time, indicating almost no redundant resources available for recovery. The results for RPD^C in Table 10 indicate that the resource transfer strategy more effectively rationalizes resource allocation, leading to further optimization of the bottleneck workstation's makespan for most instances. However, for some instances with task counts of 10, 20, and 30, RReGA's cycle time optimization is worse compared to that of RRGA. This suggests that, when the initial resource allocation is already optimal, direct optimization of each workstation's makespan can achieve more optimal cycle time, and resource transfer might degrade the allocation. Notably, when the initial resource allocation is optimal, executing resource transfer does not necessarily alter the final allocation. Resources transferred between workstations may converge back to the initial optimal allocation, but the extensive cost incurred in optimizing suboptimal allocations result in a less effective optimization compared to the non-transfer RRGA algorithm. Fig. 8 shows boxplots comparing the two algorithms on cycle time C, demonstrating that RReGA outperforms RRGA across all instance sizes, with more pronounced differences when resources are insufficient. This highlights the contribution of the resource transfer strategy to effective resource allocation and cycle time optimization.

Table 10
Results of ablation study.

Instance	CPU time (s)			RPD^C (%)		RPD^R (%)	
	RReGA	RRGA	RTGA	RReGA	RRGA	RReGA	RTGA
5–0	7.06	4.79	5.04	**0.000**	2.288	**0.000**	6.271
5–1	8.76	4.23	7.11	**0.000**	5.927	**0.000**	0.762
7–0	11.92	8.97	8.34	**0.000**	3.747	**0.000**	11.389
7–1	12.65	7.11	10.06	**0.000**	8.029	**0.000**	0.000
10–0	30.06	17.83	23.17	**0.292**	2.341	**0.000**	9.231
10–1	40.55	15.71	35.70	**0.085**	10.209	**0.000**	0.241
15–0	91.51	40.04	69.98	**0.000**	1.849	**0.000**	23.278
15–1	92.67	29.78	83.54	**0.000**	9.928	**0.000**	0.376
20–0	120.12	101.36	77.66	**0.268**	7.193	**0.000**	32.545
20–1	126.97	72.79	114.20	**0.479**	9.256	**0.000**	0.826
30–0	291.57	227.65	214.27	**0.175**	6.233	**0.000**	31.681
30–1	278.64	149.68	263.46	**0.000**	7.959	**0.000**	0.245

Table 11
Comparison results with two meta-heuristics.

Instance	CPU time (s)			RPD^O (%)		
	RReGA	HGA-PS	MVNSH	RReGA	HGA-PS	MVNSH
5–0	7.06	9.43	2.18	**0.023**	4.131	22.161
5–1	8.76	9.60	1.68	**0.004**	10.848	49.403
7–0	11.92	14.71	3.61	**0.000**	1.290	8.886
7–1	12.65	14.30	3.30	**0.000**	8.029	51.044
10–0	30.06	22.34	7.62	**0.227**	1.495	14.557
10–1	40.55	21.91	4.78	**0.000**	20.644	46.810
15–0	91.51	39.74	17.11	**0.000**	0.786	9.626
15–1	92.67	37.96	12.17	**0.000**	19.415	34.928
20–0	120.12	60.97	25.73	**0.249**	2.147	10.864
20–1	126.97	57.69	13.20	**0.000**	23.343	50.466
30–0	291.57	127.91	55.12	**0.163**	0.577	6.760
30–1	278.64	113.45	35.41	**0.000**	24.250	43.207

The resource recovery strategy aims to avoid redundant resource input, especially when total resources are excessive. Results in Table 10 and Fig. 9(a) confirm this goal. In instances of 20 and 30 tasks with sufficient resource, the removal of resource recovery results in over 30% of resource wastage. When the total worker resources are insufficient, as shown in Fig. 9(b), the performance of the algorithm without the resource recovery module is essentially consistent with that of the proposed algorithm in terms of resource consumption. Although such prominence is closely related to the total amount of resources, it underscores the significant contribution of the resource recovery strategy to reducing resource consumption.

5.3.3. Comparison with existing methods

This section systematically evaluates the performance differences between the proposed RReGA algorithm and the state-of-the-art HGA-PS [41] and MVNSH [42] algorithms in solving MRCMPSP under resource dedication policy. However, it is noteworthy that existing literature primarily focuses on MRCMPSP variants under resource sharing policy, while resource dedication policy fundamentally alter the feasible solution space. To ensure fair comparison, we adaptively modified the baseline algorithms without altering their core frameworks: (1) During individual encoding, a new resource allocation chromosome is generated to represent the number of workers allocated to each workstation; (2) During individual decoding, the task list **p** and mode list **m** for each workstation are decoded under corresponding worker quantity constraints to satisfy the resource dedication policy.

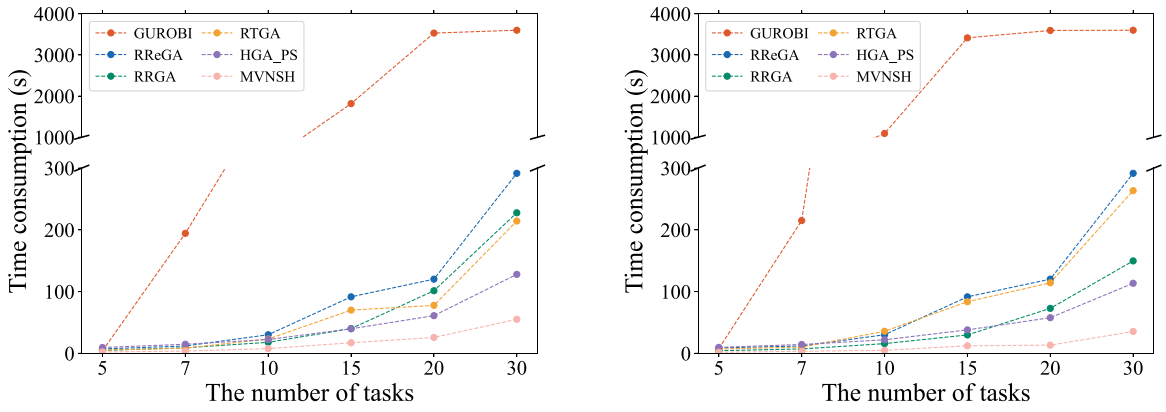
As shown in Table 11, RReGA demonstrates significant advantages across all test instances. Specifically, under resource-sufficient scenarios, RReGA achieves average optimization objective improvements of 1.74% and 12.14% over HGA-PS and MVNSH, respectively. While under resource-insufficient scenarios, these improvements further increase to 17.75% and 45.97%. The experimental results indicate that HGA-PS can achieve cycle times C close to those of RReGA when resources are sufficient, but still exhibits substantial optimization potential in resource utilization. Meanwhile, MVNSH suffers from low search efficiency under complex constraints and tends to fall into local optima. Under insufficient worker availability, RReGA demonstrates markedly superior performance compared to both baseline algorithms. This further confirms the effectiveness and superiority of the resource transfer strategy in RReGA for resource allocation under worker shortage conditions.

Fig. 10 illustrates the average computational time of RReGA and comparison algorithms across different-scale instances. It reveals two critical observations: (1) GUROBI exhibits substantially higher computational time than all meta-heuristic methods, validating the necessity of employing heuristic approaches; (2) The shorter computational times of HGA-PS and MVNSH reflect their premature convergence characteristics, indicating deficiencies in global optimization capabilities.

5.4. Study on an unbalanced case

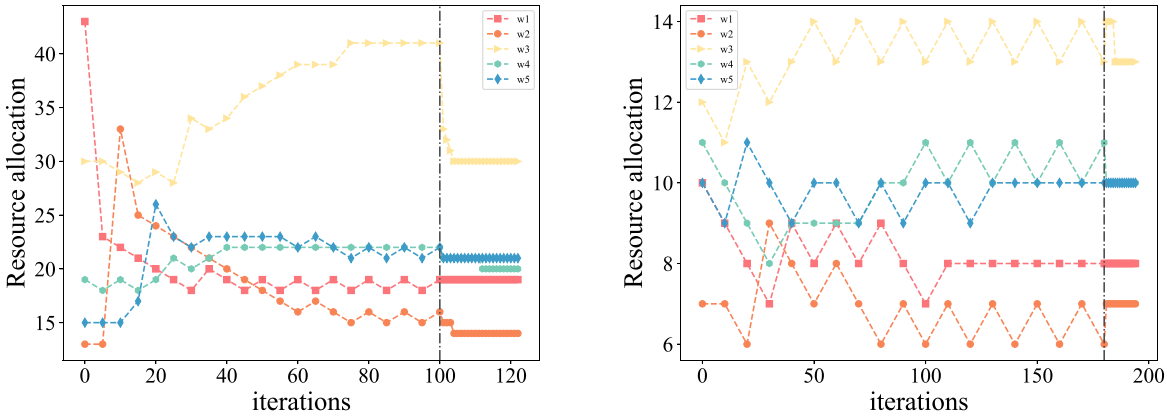
In practical production, it is challenging to ensure that each workstation is assigned an equal number of assembly tasks. This imbalance often leads to some workstations experiencing idle time, resulting in resource wastage and project delays. Evaluating whether the completion times of all workstations in unbalanced instances can be balanced by optimizing resource allocation is crucial for assessing the robustness and adaptability of the algorithm. We generated an unbalanced assembly task instance with workstation task quantities of $\{20, 20, 25, 25, 30\}$, total worker numbers of 120 and 50, and a population size of 40, with other parameters remaining consistent.

Fig. 11 depicts the entire resource reallocation process of the RReGA algorithm. The left side of the black dotted–dashed line represents the resource transfer process, while the right side represents the resource recovery process. During the resource transfer process, the resource input of the bottleneck workstation increases, a phenomenon more evident with sufficient resources. This indicates that other non-bottleneck workstations continuously transfer resources to the bottleneck workstation to optimize the cycle time C, which depends on the makespan of the bottleneck workstation. During the resource recovery stage, workstations generally have no redundant resources to reclaim in terms of insufficient resources. While the bottleneck workstation reclaims a significant amount of resources in terms of sufficient resources. Fig. 12 shows the update process of each workstation's makespan. According to Fig. 12(a), the update process of each workstation's makespan shows that the yellow broken line (bottleneck workstation's makespan) no longer changes beyond 30 iterations, while other workstations continue to transfer resources to it (as shown in Fig. 11(a)), leading to accumulation of redundant resources at the workstation w_3. At the end of the resource transfer stage, the optimal resource allocation derived by the updated resource-makespan mappings allows the bottleneck workstation w_3 to offload a large amount of redundant resources instantly.



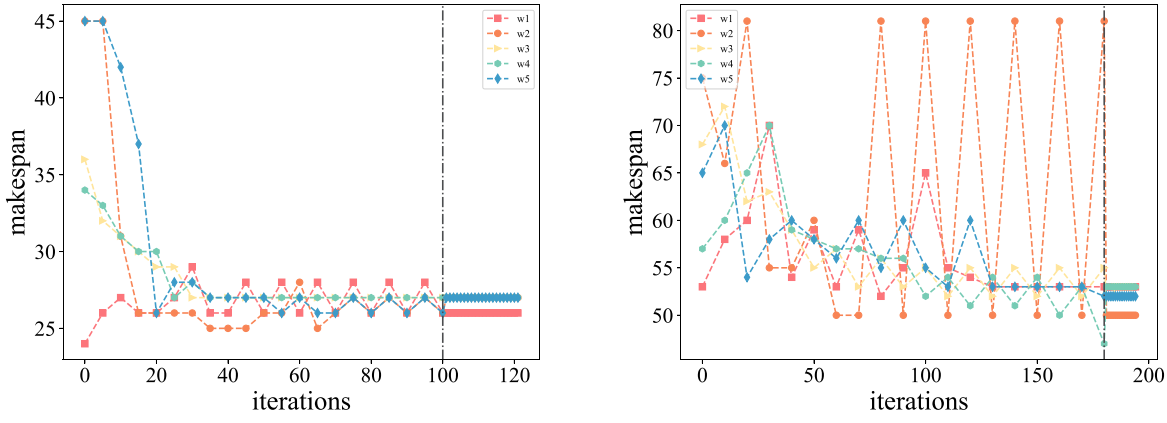
(a) sufficient resource (b) insufficient resource

Fig. 10. Time consumption of different methods.



(a) sufficient resource (b) insufficient resource

Fig. 11. The optimization process of resource allocation.



(a) sufficient resource (b) insufficient resource

Fig. 12. The optimization process of makespans of workstations.

As shown in Fig. 12(b), it can be observed that workstations experience more drastic changes in makespan on the second-half resource transfer iterations in terms of insufficient resources. It is the case because as resource input decreases, the makespan presents steep growth according to the resource-makespan mapping. The final makespans of workstations under the two resource input conditions are $\{26, 27, 27, 27, 27\}$ and $\{53, 50, 53, 53, 52\}$, respectively. This indicates that the RReGA algorithm can maintain a high balance of workstation makespan while pursuing the optimal cycle time C in unbalanced assembly task instances.

Fig. 13 illustrates the update process of the primary objective under two types of resource input. The blue curve, bearing sufficient resource input with shorter update period ($T^{rt} = 5$, each dot represents a resource-transfer update), displays fewer updates before converging

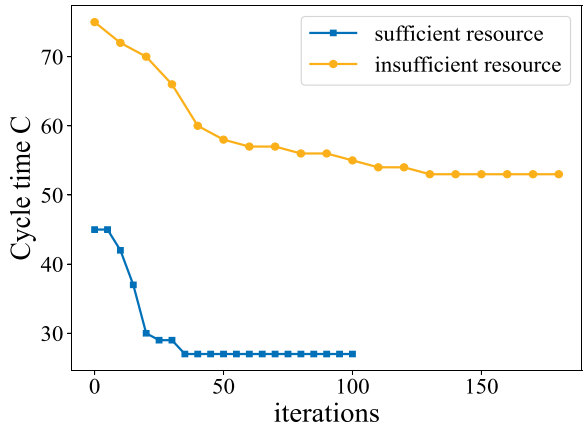


Fig. 13. The optimization process of cycle time.

to the optimal value. While the yellow curve, bearing insufficient resource input with longer update period ($T^{rt} = 10$), presents more updates to converge to the optimal value. This is due to the fact that multiple distinct encoding solutions can achieve the same makespan with sufficient resources, but not with insufficient resources. It implies that, given the same size of the encoding solution space, there are more viable encoding solutions capable of achieving the optimal makespan when resources are sufficient. Consequently, this creates a significant disparity in search efficiency between sufficient resource and insufficient resource scenarios.

6. Conclusions

This study investigates a lexicographic bi-objective optimization problem for resource allocation and task scheduling in the AFAL, which aims to minimize cycle time and resource consumption while balancing workstation completion times. A lexicographic bi-objective optimization integer programming model is established, and an efficient RReGA approach is proposed to achieve these goals. The core mechanism of this method is an embedded resource reallocation to adjust and optimize resource allocation. The resource reallocation consists of two stages: resource transfer and resource recovery. The first stage optimizes cycle time by distinguishing between bottleneck and non-bottleneck workstations based on the current optimal cycle time, following a novel resource transfer principle to assist bottleneck workstations. The second stage optimizes resource input by recovering redundant resources from workstations, ensuring each workstation meets the cycle time constraint with minimal resource consumption. The resource recovery strategy is particularly effective in cases of overall worker surplus. Additionally, an updatable resource-makespan mapping based on hybrid dispatch rules is developed to provide initial allocation, guide resource transfer, and calculate optimal resource allocation and cycle time.

Extensive experimental results validate the effect and performance of each algorithm module. In the mapping evaluation experiments, the efficiency and accuracy of the mapping estimation method are demonstrated. In comparison experiments with GUROBI, the RReGA method shows superior efficiency and optimization results in cycle time and resource input. To thoroughly demonstrate the effect and differences of resource transfer and resource recovery strategies, comprehensive ablation experiments are conducted. In an unbalanced case, the experimental results of highly consistent makespan across workstations indicate the RReGA method's optimization capability for unbalanced instances.

The flexibility of execution modes, dedication of worker resources within cycles, synchronous optimization of cycle time and resource consumption, and balance of workstation makespan considered in this study are common complex characteristics in actual AFAL scenarios. Future research could further consider the allocation of assembly tasks, which makes the entire production problem more systematic.

CRediT authorship contribution statement

Xiong Zheng: Writing – original draft, Methodology, Formal analysis, Conceptualization. **Fei Qiao:** Writing – review & editing, Funding acquisition. **Shiqi You:** Writing – review & editing. **Xi Vincent Wang:** Writing – review & editing, Supervision. **Lihui Wang:** Writing – review & editing, Supervision. **Junkai Wang:** Writing – review & editing, Validation, Supervision, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Key R&D Program of China under Grant 2022YFB3305902, and in part by the National Natural Science Foundation of China under Grant 62133011, 62203336, 62273260.

Appendix. Equivalence between Eq. (1) (2) and (13)

To demonstrate why assigning a weight of $1/n$ to the second objective (2) ensures equivalence between the weighted objective (13) and the lexicographic optimization objectives (1)(2), we proceed as follows:

Consider two distinct feasible solutions π_1 and π_2, with their corresponding values for objective (1) being C_1 and C_2, and their values for objective (2) being $\sum_{i=1}^{k} n_i^1$ and $\sum_{i=1}^{k} n_i^2$, respectively. To prove the equivalence between objectives (1) and (13)(2), we first demonstrate sufficiency:

Sufficiency Proof: Assume π_1 dominates π_2 under lexicographic optimization. By definition, we have $C_1 \le C_2$:

Case 1: $C_1 = C_2$

Lexicographic dominance requires:

$$\sum_{i=1}^{k} n_i^1 < \sum_{i=1}^{k} n_i^2 \Rightarrow \sum_{i=1}^{k} \frac{n_i^1}{n} < \sum_{i=1}^{k} \frac{n_i^2}{n}$$

Thus, the weighted objective satisfies:

$$C_1 + \sum_{i=1}^{k} \frac{n_i^1}{n} < C_2 + \sum_{i=1}^{k} \frac{n_i^2}{n}$$

Confirming π_1's dominance under objective (13).

Case 2: $C_1 < C_2$

Since C is an integer, $C_1 \le C_2 - 1$. The composite objective (13) for π_1 becomes:

$$C_1 + \sum_{i=1}^{k} \frac{n_i^1}{n} \le C_2 - 1 + \sum_{i=1}^{k} \frac{n_i^1}{n}$$

Given $0 < \sum_{i=1}^{k} \frac{n_i^1}{n} \le 1$:

$$C_1 + \sum_{i=1}^{k} \frac{n_i^1}{n} \le C_2$$

Meanwhile, π_2's composite value is:

$$C_2 + \sum_{i=1}^{k} \frac{n_i^2}{n} \ge C_2$$

Hence:

$$C_1 + \sum_{i=1}^{k} \frac{n_i^1}{n} \le C_2 + \sum_{i=1}^{k} \frac{n_i^2}{n}$$

This preserves lexicographic dominance in the weighted objective. Then we demonstrate necessity:

Necessity Proof:

Assume π_1 dominates π_2 under objectives (13), we have:

$$C_1 + \sum_{i=1}^{k} \frac{n_i^1}{n} \le C_2 + \sum_{i=1}^{k} \frac{n_i^2}{n}$$

Case 1: $C_1 = C_2$

Then:

$$\sum_{i=1}^{k} \frac{n_i^1}{n} < \sum_{i=1}^{k} \frac{n_i^2}{n} \Rightarrow \sum_{i=1}^{k} n_i^1 < \sum_{i=1}^{k} n_i^2$$

Thus, π_1 dominates π_2 in objective (2) while $C_1 = C_2$, satisfying lexicographic order.

Case 2: $C_1 < C_2$

Directly satisfies π_1's dominance in objective (1).

Case 3: $C_1 > C_2$ (Contradiction Analysis)

Then:

$$1 \le C_1 - C_2 < \sum_{i=1}^{k} \frac{n_i^2}{n} - \sum_{i=1}^{k} \frac{n_i^1}{n}$$

However, since $0 < \sum_{i=1}^{k} \frac{n_i^1}{n}, \sum_{i=1}^{k} \frac{n_i^2}{n} \le 1$:

$$-1 < \sum_{i=1}^{k} \frac{n_i^2}{n} - \sum_{i=1}^{k} \frac{n_i^1}{n} < 1$$

This contradicts $C_1 - C_2 \ge 1$. Hence, $C_1 > C_2$ is impossible.

The equivalence between the lexicographic objectives (1)(2) and the weighted objective (13) is rigorously proven.

Data availability

Data will be made available on request.

References

[1] A. Caggiano, A. Marzano, R. Teti, Resource efficient configuration of an aircraft assembly line, Procedia CIRP 41 (2016) 236–241.

[2] D. Guo, Fast scheduling of human-robot teams collaboration on synchronised production-logistics tasks in aircraft assembly, Robot. Comput.-Integr. Manuf. 85 (2024) 102620.

[3] A. Gomez, J. Rios, F. Mas, A. Vizán, Method and software application to assist in the conceptual design of aircraft final assembly lines, J. Manuf. Syst. 40 (2016) 37–53.

[4] S. Hartman, D. Briskorn, A survey of variants and extensions of the resource-constrained project scheduling problem, European J. Oper. Res. 207 (1) (2010) 1–14.

[5] M. Wang, P. Zhang, G. Zhang, K. Sun, J. Zhang, M. Jin, A resilient scheduling framework for multi-robot multi-station welding flow shop scheduling against robot failures, Robot. Comput.-Integr. Manuf. 91 (2025) 102835.

[6] H. Yang, Z. Wang, Y. Gao, W. Zhou, Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA II and Q-learning algorithm, Appl. Soft Comput. 152 (2024) 111201.

[7] U. Beşikci, Ü. Bilge, G. Ulusoy, Resource dedication problem in a multi-project environment, Flex. Serv. Manuf. J. 25 (2013) 206–229.

[8] J. Józefowska, M. Mika, R. Różycki, G. Waligóra, J. Węglarz, Simulated annealing for multi-mode resource-constrained project scheduling, Ann. Oper. Res. 102 (2001) 137–155.

[9] J. Alcaraz, L. Anton-Sanchez, F.S. da Gama, Bi-objective resource-constrained project scheduling problem with time-dependent resource costs, J. Manuf. Syst. 63 (2022) 506–523.

[10] Z. Bao, L. Chen, K. Qiu, An aircraft final assembly line balancing problem considering resource constraints and parallel task scheduling, Comput. Ind. Eng. 182 (2023) 109436.

[11] B. Zhou, J. Bao, J. Li, Y. Lu, T. Liu, Q. Zhang, A novel knowledge graph-based optimization approach for resource allocation in discrete manufacturing workshops, Robot. Comput.-Integr. Manuf. 71 (2021) 102160.

[12] M. Lee, S. Behdad, X. Liang, M. Zheng, Task allocation and planning for product disassembly with human–robot collaboration, Robot. Comput.-Integr. Manuf. 76 (2022) 102306.

[13] D. Arkhipov, O. Battaïa, J. Cegarra, A. Lazarev, Operator assignment problem in aircraft assembly lines: a new planning approach taking into account economic and ergonomic constraints, Procedia CIRP 76 (2018) 63–66.

[14] Z. Lu, Y. Ren, L. Wang, H. Zhu, A resource investment problem based on project splitting with time windows for aircraft moving assembly line, Comput. Ind. Eng. 135 (2019) 568–581.

[15] Y. Ren, Z. Lu, X. Liu, A branch-and-bound embedded genetic algorithm for resource-constrained project scheduling problem with resource transfer time of aircraft moving assembly line, Optim. Lett. 14 (8) (2020) 2161–2195.

[16] P. Fang, J. Yang, Q. Liao, R.Y. Zhong, Y. Jiang, Flexible worker allocation in aircraft final assembly line using multiobjective evolutionary algorithms, IEEE Trans. Ind. Informatics 17 (11) (2021) 7468–7478.

[17] R.H. Möhring, Minimizing costs of resource requirements in project networks subject to a fixed completion time, Oper. Res. 32 (1) (1984) 89–120.

[18] K. Ağpak, H. Gökçen, Assembly line balancing: Two resource constrained cases, Int. J. Prod. Econ. 96 (1) (2005) 129–140.

[19] B. Xin, Y. Li, J. Yu, J. Zhang, An adaptive BPSO algorithm for multi-skilled workers assignment problem in aircraft assembly lines, Assem. Autom. 35 (4) (2015) 317–328.

[20] A. Biele, L. Mönch, Hybrid approaches to optimize mixed-model assembly lines in low-volume manufacturing, J. Heuristics 24 (1) (2018) 49–81.

[21] Z. Bao, L. Chen, K. Qiu, A robust optimization approach for the resource investment problem of an aircraft final assembly line, IEEE Trans. Autom. Sci. Eng. 20 (3) (2022) 2135–2146.

[22] T. Portoleau, C. Artigues, R. Guillaume, Robust decision trees for the multi-mode project scheduling problem with a resource investment objective and uncertain activity duration, European J. Oper. Res. 312 (2) (2024) 525–540.

[23] T. Borreguero Sanchidrián, T. Portoleau, C. Artigues, A. García Sánchez, M. Ortega Mier, P. Lopez, Large neighborhood search for an aeronautical assembly line time-constrained scheduling problem with multiple modes and a resource leveling objective, Ann. Oper. Res. 338 (1) (2024) 13–40.

[24] G. Heike, M. Ramulu, E. Sorenson, P. Shanahan, K. Moinzadeh, Mixed model assembly alternatives for low-volume manufacturing: the case of the aerospace industry, Int. J. Prod. Econ. 72 (2) (2001) 103–120.

[25] B.J.V. Da Silva, R. Morabito, D.S. Yamashita, H.H. Yanasse, Production scheduling of assembly fixtures in the aeronautical industry, Comput. Ind. Eng. 67 (2014) 195–203.

[26] J. Buergin, S. Helming, J. Andreas, P. Blaettchen, Y. Schweizer, F. Bitte, B. Haefner, G. Lanza, Local order scheduling for mixed-model assembly lines in the aircraft manufacturing industry, Prod. Eng. 12 (2018) 759–767.

[27] S. Shan, Z. Hu, Z. Liu, J. Shi, L. Wang, Z. Bi, An adaptive genetic algorithm for demand-driven and resource-constrained project scheduling in aircraft assembly, Inf. Technol. Manag. 18 (2017) 41–53.

[28] H. Zhu, Z. Lu, C. Lu, Y. Ren, A reactive scheduling method for disturbances in aircraft moving assembly line, Int. J. Prod. Res. 59 (15) (2021) 4756–4772.

[29] Q. Yan, H. Wang, Double-layer Q-learning-based joint decision-making of dual resource-constrained aircraft assembly scheduling and flexible preventive maintenance, IEEE Trans. Aerosp. Electron. Syst. 58 (6) (2022) 4938–4952.

[30] V. Tereshchuk, N. Bykov, S. Pedigo, S. Devasia, A.G. Banerjee, A scheduling method for multi-robot assembly of aircraft structures with soft task precedence constraints, Robot. Comput.-Integr. Manuf. 71 (2021) 102154.

[31] J. Chen, J. Zhu, D. Zhang, Multi-project scheduling problem with human resources based on dynamic programming and staff time coefficient, in: 2014 International Conference on Management Science & Engineering 21th Annual Conference Proceedings, IEEE, 2014, pp. 1012–1018.

[32] A. Can, G. Ulusoy, Multi-project scheduling with two-stage decomposition, Ann. Oper. Res. 217 (2014) 95–116.

[33] T. Wauters, J. Kinable, P. Smet, W. Vancroonenburg, G. Vanden Berghe, J. Verstichel, The multi-mode resource-constrained multi-project scheduling problem: The MISTA 2013 challenge, J. Sched. 19 (2016) 271–283.

[34] S. Asta, D. Karapetyan, A. Kheiri, E. Özcan, A.J. Parkes, Combining Monte-Carlo and hyper-heuristic methods for the multi-mode resource-constrained multi-project scheduling problem, Inform. Sci. 373 (2016) 476–498.

[35] M.J. Geiger, A multi-threaded local search algorithm and computer implementation for the multi-mode, resource-constrained multi-project scheduling problem, European J. Oper. Res. 256 (3) (2017) 729–741.

[36] U. Beşikci, Ü. Bilge, G. Ulusoy, Multi-mode resource constrained multi-project scheduling and resource portfolio problem, European J. Oper. Res. 240 (1) (2015) 22–31.

[37] Z. Bao, L. Chen, K. Qiu, A surrogate-assisted heuristic approach for the joint optimization of resource allocation and scheduling of an aircraft final assembly line, J. Manuf. Syst. 70 (2023) 99–112.

[38] A. Biele, L. Mönch, Decomposition methods for cost and tardiness reduction in aircraft manufacturing flow lines, Comput. Oper. Res. 103 (2019) 134–147.

[39] R. Raz, S. Safra, A sub-constant error-probability low-degree test, and a sub-constant error-probability PCP characterization of NP, in: Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing, 1997, pp. 475–484.

[40] S. Shadrokh, F. Kianfar, A genetic algorithm for resource investment project scheduling problem, tardiness permitted with penalty, European J. Oper. Res. 181 (1) (2007) 86–101.
[41] J.C. Chen, H.Y. Lee, W.H. Hsieh, T.L. Chen, Applying hybrid genetic algorithm to multi-mode resource constrained multi-project scheduling problems, J. Chin. Inst. Eng. 45 (1) (2022) 42–53.
[42] R.K. Chakrabortty, A. Abbasi, M.J. Ryan, Multi-mode resource-constrained project scheduling using modified variable neighborhood search heuristic, Int. Trans. Oper. Res. 27 (1) (2020) 138–167.